Reading What Matters: Table-Conditioned Selective Text Reading for Text-Tabular Prediction

Daniel Polyanovsky
Technion – Israel Institute of Technology

Abstract

Text-tabular models encode *every* text field of a row, although the structured columns often already determine the prediction and only some rows need some of their text. We propose a *reading agent*: a table encoder initialises a recurrent *anytime* reader that consumes frozen-E5 sentence items one at a time and can predict after any number of reads, and an *expected-gain head*, adapted from dynamic feature selection, reads greedily while the predicted loss reduction exceeds a cost λ. On all 20 text datasets of MulTaBench (five splits each), learned reading significantly beats a random reader with the same per-row budget at every budget we test, e.g. by +0.026 AUROC and +0.051 R^2 at a nominal 10% of the retained text items. For full-budget accuracy, a validation-tuned blend of the reader with a CatBoost Joint-Frozen model significantly improves on CatBoost Joint Frozen (+0.012 AUROC, +0.017 R^2) but does not reach the Joint TAR baselines. We report this gap, a negative result for imitation learning, and where adaptive reading does and does not help.

1 Introduction

In text-tabular learning, a row combines numerical and categorical columns with free-text fields such as reviews or job descriptions [1]. Standard practice embeds each field with a pretrained encoder such as E5 [20] and trains a tabular learner on the result (*Joint Frozen*), or fine-tunes the encoder on the target (*Joint TAR*) [2, 1]. Both treat all text as equally necessary for every row. Human analysts read differently: if a wine's price and region already pin down its grade, the review is worth reading only when the table leaves the answer ambiguous, and then one sentence may suffice. We turn this into an algorithm: the structured part of a row initialises a recurrent reader, which acquires text one *item* (a field-tagged sentence chunk) at a time, choosing the item with the highest predicted loss reduction, until that gain no longer pays for the reading cost.

Contributions. (i) A table-conditioned anytime reader for text-tabular rows, trained with random reading orders so it is valid after any subset of reads. (ii) A reading policy, label-free at inference, that regresses observed one-step loss reductions, in the spirit of DIME [6], in two access modes: *content* keys (routing over already-embedded text) and *metadata* keys (field and position only, so unread content is not inspected for selection). (iii) An evaluation on all 20 MulTaBench text datasets with paired local baselines and dataset-level statistics. (iv) Analyses of where adaptive reading helps, and a negative result for imitating a label oracle.

2 Related Work

Text-tabular learning. MulTaBench [1] curates datasets on which text and structure are jointly needed and target-aware representations help. It evaluates CatBoost [15], LightGBM [13], TabM [7] and TabPFN [10, 9] with frozen or LoRA-tuned [12] E5 embeddings. TabSTAR [2] unfreezes the

NLP 097215 Final Project (Track 1), Technion, 2026. Code: `https://github.com/htmldanielpoly/nlp_final_project`; weights: `https://huggingface.co/htmldaniel/reading-agent-multabench`.

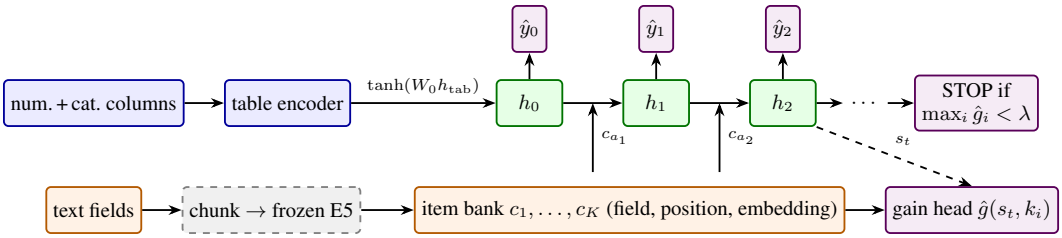


Figure 1: The reading agent: the table initialises a 2-layer GRU; at each step the gain head scores unread items, the reader consumes the best one, and $\hat{y}_t$ is available after every step.

text encoder inside a tabular foundation model, and CARTE [14] embeds column names and strings for transfer. Shi et al. [18] show that ensembling neural text models with tabular learners is strong; our blend follows that practice and is not claimed as novel. **Dynamic feature selection.** Active feature acquisition chooses which features to observe per instance [19]. Covert et al. [4] learn greedy information-maximising policies without RL, and DIME [6] regresses the reduction in prediction loss and stops at a threshold. Our gain head applies DIME's estimator to field-tagged text chunks, with a table-derived prior state. **Adaptive reading.** LSTM-Jump [21] and Skim-RNN [17] learn to skip or skim tokens of one document, and adaptive computation time [8] learns how many recurrent steps to spend; we select among a tabular row's fields and chunks, conditioned on its columns.

3 Method

Text items. Each text field is split into sentences, merged or cut into chunks of at most 48 words (a field's chunks beyond the eighth merge into one tail item, which E5 truncates at 512 tokens). Items are allocated round-robin across fields up to $K = \max(8, \#\text{fields})$ per row, so every field has at least one item; the kept items contain every word of the row on 13 of the 20 datasets, before encoder truncation (coverage: Appendix B). Each item is verbalised as `passage: <column>: <chunk>` and embedded once with frozen E5-small-v2 [20] ($e_i \in \mathbb{R}^{384}$, cached). Inside the model, an item is $c_i = \text{LN}(W_{\text{item}} e_i + W_{\text{name}}\,\text{E5}(\text{field name}) + p_{\text{pos}(i)}) \in \mathbb{R}^{128}$.

Table encoder and anytime reader. Numerical columns are quantile-normalised. Each column becomes a token $x_j w_j + m_j u_j + n_j$ (numerical; m_j marks missingness) or $\text{Emb}(v_j) + n_j$ (categorical), where n_j is a projected E5 embedding of the column name. The flattened tokens pass through an MLP to $h_{\text{tab}} \in \mathbb{R}^{128}$, which initialises both layers of a GRU [3] (Fig. 1). After reading items $a_1, \ldots, a_t$, the state is $s_t = [h_t^{(2)};\ \frac{1}{t}\sum_{\tau \le t} c_{a_\tau};\ h_{\text{tab}}]$ (the mean term is zero at $t = 0$), and an MLP predictor outputs $f(s_t)$. We train the encoder, reader and predictor together, sampling a fresh random reading order per batch:

$$\mathcal{L}_{\text{read}} = \tfrac{1}{2} \cdot \tfrac{1}{n+1} \textstyle\sum_{t=0}^{n} \ell\big(f(s_t), y\big) + \tfrac{1}{2}\,\ell\big(f(s_n), y\big), \tag{1}$$

where $n \le K$ is the row's number of items, ℓ is cross-entropy for classification and MSE on the standardised target for regression. Random orders make f a valid predictor after *any* subset of reads, which the policy relies on.

Expected-gain policy. With the reader frozen, a head $\hat{g}(s_t, k_i) = \text{MLP}([s_t; k_i])$ predicts the loss reduction from reading item i next. The *content* key is $k_i = c_i$. The *metadata* key uses only the field and position embeddings, so the content of unread text is never inspected. On a held-out inner-validation split we roll in ε-greedily ($\varepsilon = 0.3$) with the current head, in the DAgger style [16], so training states match the states the policy visits. At every visited state we compute the observed gain of every unread item and regress it:

$$\Delta_i(s_t) = \ell\big(f(s_t), y\big) - \ell\big(f(s_t \oplus c_i), y\big), \qquad \mathcal{L}_{\text{gain}} = \textstyle\sum_t \sum_{i\ \text{unread}} \big(\hat{g}(s_t, k_i) - \Delta_i(s_t)\big)^2. \tag{2}$$

The label only builds targets: at test time $\hat{g}$ estimates $\mathbb{E}[\Delta_i \mid s_t, k_i]$ without y, a predicted improvement of *our* predictor's loss rather than an exact information estimate. **Inference:** read $\arg\max_i \hat{g}$ while $\max_i \hat{g} \ge \lambda$. For a target fraction of items read (10/25/50/75%), we bisect λ on the validation split. This needs only $\hat{g}$, not labels.

Why not imitation? (negative result). Our first policy, a pointer network imitating a greedy oracle that reads the item most reducing the loss for the *true* label, was no better than random in pilots:

Table 1: Full-budget accuracy, dataset means (folds averaged per dataset). W/T/L: datasets where the Blend wins, ties or loses against that row; *: two-sided Wilcoxon $p < 0.05$ (local rows only). $^{\dagger n}$: reference for only n datasets; [ ]: the Blend's mean on those n.

	Classification (10, AUROC)		Regression (10, R^2)	
Method	mean	Blend W/T/L	mean	Blend W/T/L
Ours				
Reader, table only (0 reads)	0.751	10/0/0*	0.422	10/0/0*
Reader, read all	0.893	10/0/0*	0.640	10/0/0*
Blend: CatBoost Frozen + reader	0.900		0.702	
Local baselines (identical rows)				
CatBoost, tabular only	0.762	10/0/0*	0.430	10/0/0*
CatBoost, Joint Frozen	0.887	10/0/0*	0.684	8/1/1*
MulTaBench reference, Joint Frozen (E5)				
CatBoost, Joint Frozen	0.889	10/0/0	0.693	9/0/1
LightGBM, Joint Frozen	0.872	10/0/0	0.675	10/0/0
TabM, Joint Frozen	0.893	7/0/3	0.721	3/0/7
TabPFN-2.5, Joint Frozen	0.885$^{\dagger 8}$ [0.881]	2/0/6	0.727	0/0/10
MulTaBench reference, Joint TAR (LoRA-tuned E5)				
CatBoost, Joint TAR	0.907	3/0/7	0.716	2/0/8
LightGBM, Joint TAR	0.894	7/0/3	0.702	6/0/4
TabM, Joint TAR	0.911	2/0/8	0.728	1/0/9
TabPFN-2.5, Joint TAR	0.900$^{\dagger 8}$ [0.881]	0/0/8	0.732	0/0/10

plausibly because the oracle's choices depend on y (it reached 0.966 AUROC on Women Clothing with 1.5 reads per row), which the policy cannot see; regressing the *expected* gain avoids this.

Full-budget accuracy mode. For the benchmark objective we combine the read-all reader with a CatBoost Joint-Frozen model built from the same E5 cache (structured columns + PCA-30 of each field's whole-text E5 embedding). The convex weight $w \in \{0, 0.05, \ldots, 1\}$ is tuned on inner validation. The reader learns target-trained projections of the text embeddings, so we treat **Joint TAR as the relevant bar** and also report Joint Frozen.

4 Experiments

Setup. We use all 20 MulTaBench text datasets (10 classification, binary or macro one-vs-rest AUROC; 10 regression, R^2) under the paper protocol: random holdout splits (seed $42{+}$fold, test capped at 2,000 and train at 10,000 rows) ported from the official `paper_version` code; five splits per dataset. An inner 80/20 split of the outer training set provides validation for early stopping (100 epochs, patience 15), the gain head, λ and the blend weight; pilots informed the architecture, but the recipe was fixed before the five-split evaluation. All datasets share one configuration ($d = 128$, dropout 0.2, AdamW lr 10^{-3}, batch 256; gain head 15 epochs, lr $3 \cdot 10^{-3}$). Local CatBoost baselines share our exact rows; external references are the official leaderboard scores for the same split seeds (row identity unverified, so they are approximate references without significance tests). Following Demšar [5], we average folds within each dataset and run two-sided Wilcoxon signed-rank tests over datasets, separately for AUROC and R^2.

Full-budget accuracy (Table 1). The blend improves on CatBoost Joint Frozen on both families, against our local run on identical rows (+0.012 AUROC with 10/10 wins, $p = 0.002$; +0.017 R^2 with 8/10 wins, $p = 0.008$); against the leaderboard's CatBoost Frozen reference it wins on 10/10 and 9/10 datasets. Without fold 0, which pilots also used, the gains are +0.013 and +0.017 (both $p < 0.01$). It does *not* reach Joint TAR (-0.008 AUROC and -0.014 R^2 against CatBoost TAR; TabM and TabPFN-2.5 are stronger still), so the objective of beating Joint TAR is **not met**. The reader alone is competitive with CatBoost Frozen on classification (+0.005 AUROC) but weaker on regression (-0.044 R^2); their errors differ, which is why the blend helps (per-dataset results: Appendix A). The reader wins clearly when text carries most of the signal (Video Games R^2 0.366 vs. 0.295 for CatBoost Frozen; Jigsaw AUROC 0.868 vs. 0.819) and loses most on Zomato (0.608 vs. 0.825), whose rows average 1,900 words, of which the K items keep only 6%.

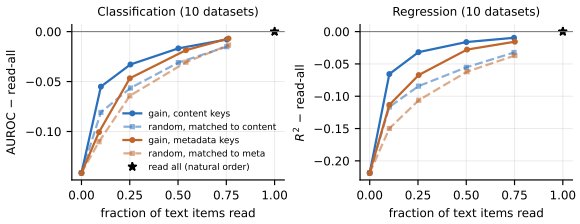


Figure 2: Score minus read-all score (dataset means) vs. fraction of items read. Solid: gain policy; dashed: random reader with the same per-row reads (3 orders); ⋆: read-all, natural order.

Table 2: Gain policy minus matched-random reader per nominal budget (dataset level). "read": mean achieved test fraction; p_{Holm}: Wilcoxon p, Holm-adjusted [11] over the 8 key×budget cells per family.

		Classification (ΔAUROC)				Regression (ΔR^2)			
Keys	Budget	read	Δ	W/T/L	p_{Holm}	read	Δ	W/T/L	p_{Holm}
content	10%	10%	+0.026	9/1/0	0.031	10%	+0.051	10/0/0	0.016
content	25%	25%	+0.024	9/1/0	0.031	25%	+0.053	10/0/0	0.016
content	50%	50%	+0.014	9/1/0	0.031	50%	+0.039	10/0/0	0.016
content	75%	75%	+0.007	9/1/0	0.031	75%	+0.023	9/0/1	0.018
metadata	10%	9%	+0.010	8/1/1	0.031	10%	+0.037	10/0/0	0.016
metadata	25%	25%	+0.018	8/1/1	0.031	25%	+0.039	9/0/1	0.018
metadata	50%	54%	+0.012	8/1/1	0.031	50%	+0.034	9/0/1	0.018
metadata	75%	76%	+0.007	9/1/0	0.031	75%	+0.021	9/0/1	0.016

Selective reading (Fig. 2, Table 2). At 10% of the items the content policy beats the matched random reader by +0.026 AUROC (9/10 datasets) and +0.051 R^2 (10/10); all 16 key×budget comparisons are significant after Holm correction. The random reader gets the policy's per-row read counts, isolating *which* items are read (and in what order) from *how many*. Against random items with the same *total* reads spread over rows by item count (Appendix C), the 10% gap widens to +0.053 AUROC and +0.122 R^2 (10/10 and 10/10): at small budgets choosing *which rows* to read matters as much as which items, and stored field order is barely better than random. Metadata keys, which never see unread content, also win at every budget with smaller margins (+0.010 AUROC, +0.037 R^2 at 10%). Discrete gain scores make some nominal budgets unattainable (Table 2); only metadata keys could avoid embedding unread items.

Where reading helps. At 10% of the items the content policy recovers a median 63% of what reading everything adds over the table alone, against 39% for the matched random reader (metadata keys: 45% vs. 24%; 19 datasets where text adds ≥0.02, Wilcoxon p<0.001). The advantage grows with how much the text helps (Spearman $\rho = 0.75$, $p = 0.013$ on regression; $\rho = 0.50$, $p = 0.138$ on classification), rather than simply with text length: it is largest on Rotten Tomatoes (+0.14 R^2), Vancouver Salaries (+0.11) and DS Salary (+0.085 AUROC), and near zero where the table decides (Montgomery Salaries) or rows hold 1–2 items (Product Sentiment, Jigsaw). Giving the table encoder each field's presence and log length (Appendix C) raises table-only scores (+0.025 AUROC, +0.037 R^2) and shrinks the 10% content advantage on classification (+0.028→+0.017) but not regression (+0.051→+0.064): reading is partly, not only, "read the long fields".

5 Conclusion

On 20 MulTaBench text datasets, table-conditioned selective reading with a DIME-style gain policy significantly beats budget-matched random reading at every budget, even with metadata-only keys; blended with CatBoost Joint Frozen, the reader improves on it but does not reach Joint TAR. **Limitations:** a simple MLP table encoder holds back the full-budget reader (weak on regression); the gain head is one-step greedy; the inner validation split is reused for stopping, gain fitting and calibration; the K-item cap drops long text (Zomato). **Future work:** a stronger table backbone (e.g. TabM), lexical previews before E5 encoding, and LoRA-tuned item encoders.

References

[1] Alan Arazi, Eilam Shapira, Shoham Grunblat, Mor Ventura, Elad Hoffer, Gil Blayer, David Holzmüller, Lennart Purucker, Gaël Varoquaux, Frank Hutter, and Roi Reichart. MulTaBench: Benchmarking multimodal tabular learning with text and image. *arXiv preprint arXiv:2605.10616*, 2026.

[2] Alan Arazi, Eilam Shapira, and Roi Reichart. TabSTAR: A tabular foundation model for tabular data with text fields. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, 2025.

[3] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014.

[4] Ian Covert, Wei Qiu, Mingyu Lu, Na Yoon Kim, Nathan White, and Su-In Lee. Learning to maximize mutual information for dynamic feature selection. In *International Conference on Machine Learning (ICML)*, 2023.

[5] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[6] Soham Gadgil, Ian Covert, and Su-In Lee. Estimating conditional mutual information for dynamic feature selection. In *International Conference on Learning Representations (ICLR)*, 2024.

[7] Yury Gorishniy, Akim Kotelnikov, and Artem Babenko. TabM: Advancing tabular deep learning with parameter-efficient ensembling. *arXiv preprint arXiv:2410.24210*, 2024.

[8] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[9] Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, Brendan Roof, Benjamin Jäger, Dominik Safaric, Simone Alessi, Adrian Hayler, et al. TabPFN-2.5: Advancing the state of the art in tabular foundation models. *arXiv preprint arXiv:2511.08667*, 2025.

[10] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025.

[11] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[13] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[14] Myung Jun Kim, Léo Grinsztajn, and Gaël Varoquaux. CARTE: Pretraining and transfer for tabular learning. In *International Conference on Machine Learning (ICML)*, 2024.

[15] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

[16] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[17] Minjoon Seo, Sewon Min, Ali Farhadi, and Hannaneh Hajishirzi. Neural speed reading via Skim-RNN. In *International Conference on Learning Representations (ICLR)*, 2018.

[18] Xingjian Shi, Jonas Mueller, Nick Erickson, Mu Li, and Alexander J. Smola. Benchmarking multimodal AutoML for tabular data with text fields. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.

[19] Hajin Shim, Sung Ju Hwang, and Eunho Yang. Joint active feature acquisition and classification with variable-size set encoding. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

[20] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

[21] Adams Wei Yu, Hongrae Lee, and Quoc Le. Learning to skim text. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.

A Per-dataset results

Table 3: Fold-mean scores per dataset. CB: our CatBoost (tab: tabular only). R: reader (table only / gain policy, content keys, @25% / read all). Blend ± fold std. LB: MulTaBench leaderboard for matching split seeds (row identity unverified); best TAR is the maximum over the four learners. Bold: best of our columns.

Dataset	folds	CB tab	CB Frozen	R: table	R: gain@25%	R: all	Blend	LB CB Frozen	LB CB TAR	LB best TAR
Classification (AUROC)										
Fake Job Posting	5	0.679	0.893	0.672	0.851	0.901	**0.911**±0.019	0.906	0.912	0.929
Jigsaw Toxicity	5	0.566	0.819	0.551	0.760	0.868	**0.870**±0.021	0.818	0.927	0.937
Kickstarter	5	0.654	0.729	0.646	0.711	0.734	**0.744**±0.012	0.729	0.744	0.752
DS Salary	5	0.554	0.822	0.553	0.798	0.826	**0.831**±0.006	0.820	0.828	0.839
Michelin	5	0.874	0.906	0.867	0.894	0.904	**0.912**±0.004	0.905	0.917	0.925
Product Sentiment	5	0.826	0.893	0.819	0.855	0.904	**0.906**±0.005	0.900	0.914	0.925
Spotify Genres	5	0.922	0.960	0.912	0.946	0.957	**0.967**±0.002	0.960	0.964	0.964
US Accidents	5	0.903	0.971	0.861	0.952	0.957	**0.972**±0.005	0.971	0.977	0.982
Wine Review	5	0.857	0.975	0.856	0.955	0.977	**0.979**±0.001	0.975	0.981	0.982
Women Clothing	5	0.780	0.903	0.773	0.874	0.898	**0.905**±0.003	0.903	0.909	0.913
Regression (R^2)										
Babies Prices	5	0.385	0.866	0.399	0.786	0.824	**0.875**±0.038	0.872	0.893	0.927
Book Price	5	0.212	0.517	0.191	0.305	0.364	**0.519**±0.032	0.543	0.565	0.565
Book Readability	5	0.782	0.815	0.790	0.811	0.820	**0.831**±0.018	0.819	0.829	0.844
Mercari	5	0.099	0.433	0.097	0.332	0.374	**0.444**±0.026	0.436	0.450	0.471
Montgomery Sal.	5	0.974	**0.976**	0.961	0.963	0.964	0.975±0.007	0.972	0.975	0.984
Rotten Tomatoes	5	0.190	0.493	0.183	0.462	0.497	**0.524**±0.030	0.508	0.521	0.573
SCImago Impact	5	0.737	0.870	0.730	0.841	0.854	**0.873**±0.007	0.868	0.875	0.888
Vancouver Sal.	5	0.314	0.754	0.306	0.659	0.732	**0.769**±0.028	0.764	0.788	0.798
Video Games	5	0.038	0.295	0.036	0.333	0.366	**0.381**±0.130	0.325	0.419	0.506
Zomato	5	0.575	**0.825**	0.523	0.591	0.608	**0.825**±0.006	0.825	0.840	0.862

B Text coverage of the item representation

Table 4: Fold-0 test rows. Items/row: items kept after the cap K; rows over K: rows with more candidate chunks than K; words kept: share of the row's words inside kept items (an upper bound on what E5 sees, since a field's chunks beyond the eighth are merged into one and E5 truncates at 512 tokens).

Dataset	text fields	K	words/row	items/row	rows over K	words kept
Classification						
Fake Job Posting	3	8	176	5.4	15%	89%
Jigsaw Toxicity	1	8	52	1.8	0%	100%
Kickstarter	3	8	26	3.0	0%	100%
DS Salary	5	8	29	4.8	0%	100%
Michelin	6	8	110	7.4	17%	94%
Product Sentiment	1	8	18	1.0	0%	100%
Spotify Genres	3	8	9	3.0	0%	100%
US Accidents	6	8	19	6.0	0%	100%
Wine Review	2	8	41	2.2	0%	100%
Women Clothing	2	8	62	2.7	0%	100%
Regression						
Babies Prices	4	8	12	2.7	0%	100%
Book Price	5	8	176	7.4	43%	77%
Book Readability	6	8	185	7.8	49%	90%
Mercari	6	8	37	5.8	1%	99%
Montgomery Sal.	3	8	9	3.0	0%	100%
Rotten Tomatoes	13	13	71	12.8	1%	100%
SCImago Impact	10	10	28	10.0	0%	100%
Vancouver Sal.	2	8	5	2.0	0%	100%
Video Games	2	8	6	2.0	0%	100%
Zomato	7	8	1909	7.8	79%	6%

C Reading controls

Table 5: Dataset-level mean difference (wins/datasets) at nominal budgets 10% and 50%, all five folds (folds averaged per dataset). *Same total reads*: random items, with the policy's total number of reads spread over rows in proportion to their item counts. *Same per-row reads*: random items with the policy's per-row counts (as in Table 2). *Row allocation*: the second random reader minus the first. *Natural order*: fields and chunks in their stored order, same counts as the first reader. *: Holm-adjusted Wilcoxon $p < 0.05$ over the eight key×budget cells.

		Classification (ΔAUROC)		Regression (ΔR^2)	
Comparison	Keys	10%	50%	10%	50%
gain − random (same total reads)	content	+0.053 (10/10)*	+0.019 (10/10)*	+0.122 (10/10)*	+0.068 (10/10)*
	metadata	+0.020 (8/10)	+0.015 (8/10)	+0.086 (10/10)*	+0.060 (10/10)*
gain − random (same per-row reads)	content	+0.027 (9/10)*	+0.014 (9/10)*	+0.051 (10/10)*	+0.042 (10/10)*
	metadata	+0.014 (9/10)*	+0.013 (8/10)*	+0.038 (10/10)*	+0.038 (10/10)*
policy row allocation − uniform	content	+0.027 (10/10)*	+0.006 (7/10)	+0.071 (10/10)*	+0.027 (7/10)
	metadata	+0.005 (8/10)	+0.003 (4/10)	+0.048 (10/10)*	+0.022 (7/10)
gain − natural field order	content	+0.044 (10/10)*	+0.007 (7/10)	+0.123 (10/10)*	+0.086 (10/10)*
	metadata	+0.009 (6/10)	+0.003 (4/10)	+0.088 (10/10)*	+0.079 (8/10)

Table 6: Adding each text field's presence flag and log word count to the table encoder (fold 0, matched datasets). Gain $-$ random is the advantage over the per-row matched random reader. *: Wilcoxon $p < 0.05$.

	Classification (AUROC)			Regression (R^2)		
	without	with	Δ (W/n)	without	with	Δ (W/n)
Reader, table only	0.753	0.778	+0.025 (9/10)*	0.430	0.467	+0.037 (8/10)*
Reader, read all	0.889	0.887	$-$0.002 (3/10)	0.671	0.645	$-$0.026 (2/10)
gain $-$ random, content @10%	+0.028	+0.017	$-$0.011 (1/10)*	+0.051	+0.064	+0.013 (8/10)
gain $-$ random, content @25%	+0.023	+0.017	$-$0.006 (1/10)*	+0.054	+0.065	+0.010 (4/10)
gain $-$ random, metadata @10%	+0.016	+0.009	$-$0.007 (1/10)*	+0.034	+0.028	$-$0.006 (5/10)
gain $-$ random, metadata @25%	+0.018	+0.015	$-$0.002 (3/10)	+0.040	+0.023	$-$0.017 (2/10)*